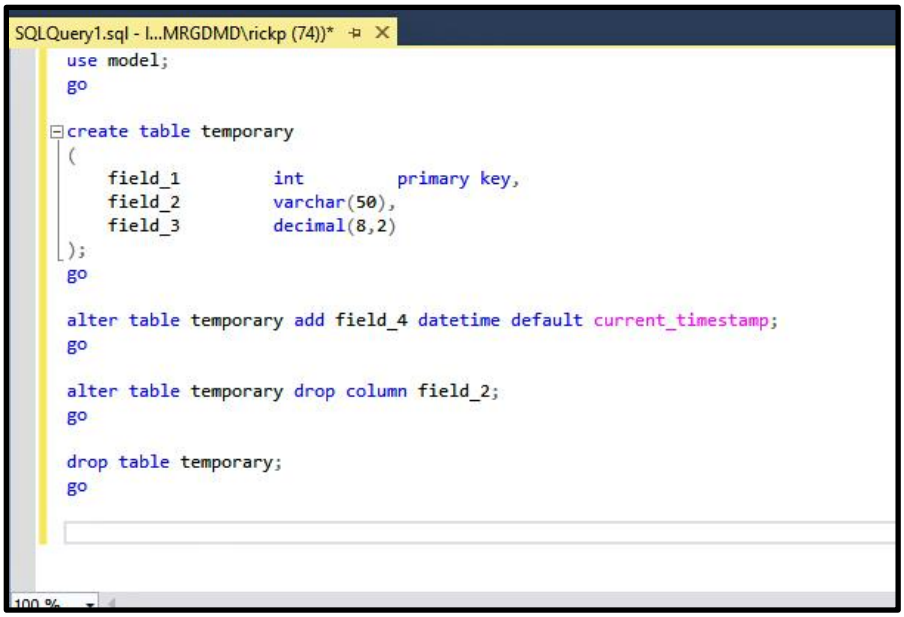
Chapter 19B: SQL

In our previous lesson we learned how to define a table within a schema and how to insert data into that table. We also took a look at how to create a primary key, foreign keys and nullable fields.

Now it's time to move on and learn how to alter and drop tables as well as update, delete and select information. Let's jump to the inside and take a look.

My alter and drop table example is shown here. (Figure 19B.1) Please note that in most groups only one person, the database designer, is allowed to create, alter or drop tables to ensure the accuracy of the design. However, for those of you taking a certification test this syntax is important because you can bet it will be on your test.

Figure 19B.1: Alter and Drop Table



Okay, now that we have all the DDL syntax under our belts we can move onto some select statements. Select statements are also known as queries. Queries return rows of data from the database. Our first, most elementary example, is shown here. (Figure 19B.2)

In this example we have asked the database to show us the Instructor_ID, First_Name, Last_Name and Department columns from the Instructors table. The database responds by displaying this information for all rows in the Instructors table.

Please note that we have omitted the Department_Head field from this query. The rule for queries is simple, you may request any combination of fields and those fields may be listed in any order. (Figure 19B.3)

Figure 19B.2: Simple Select

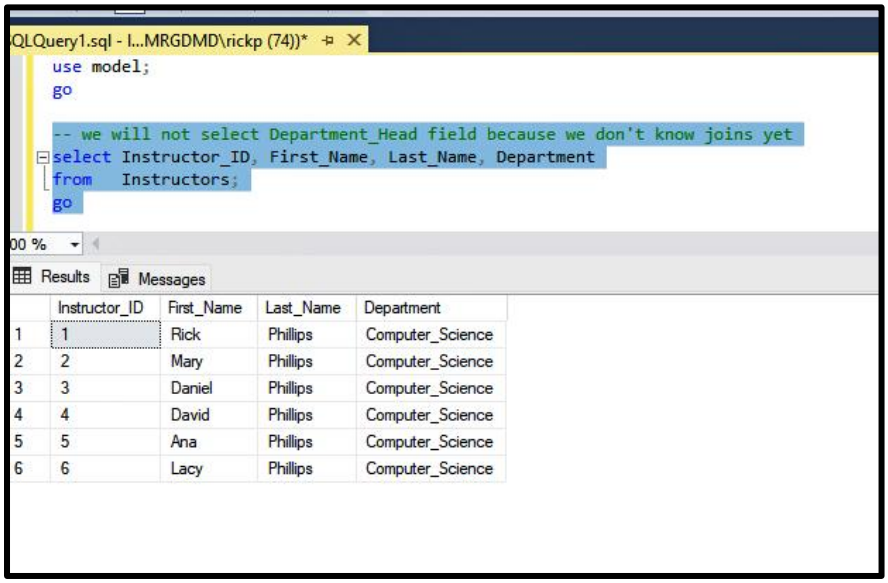
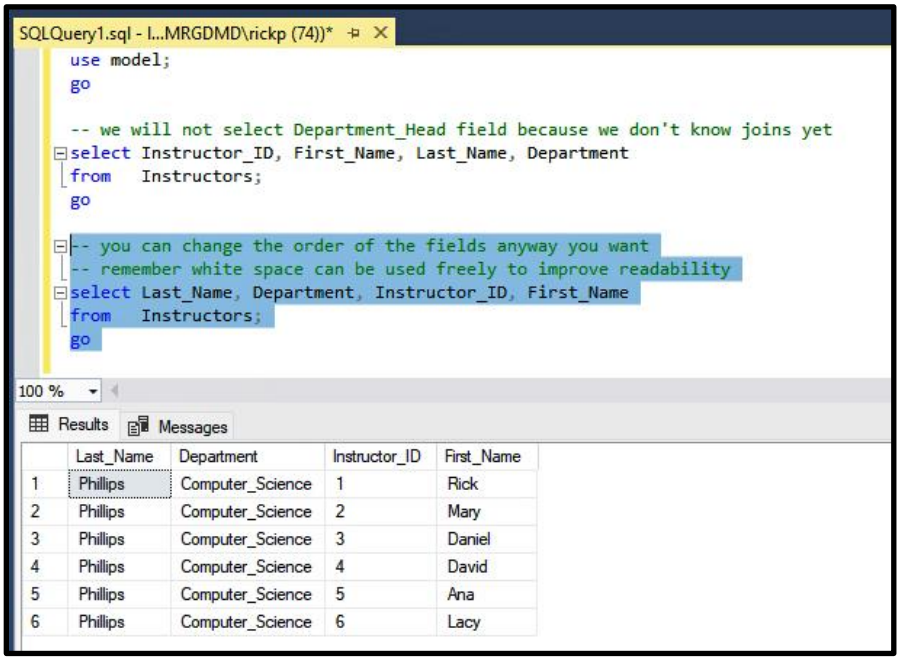
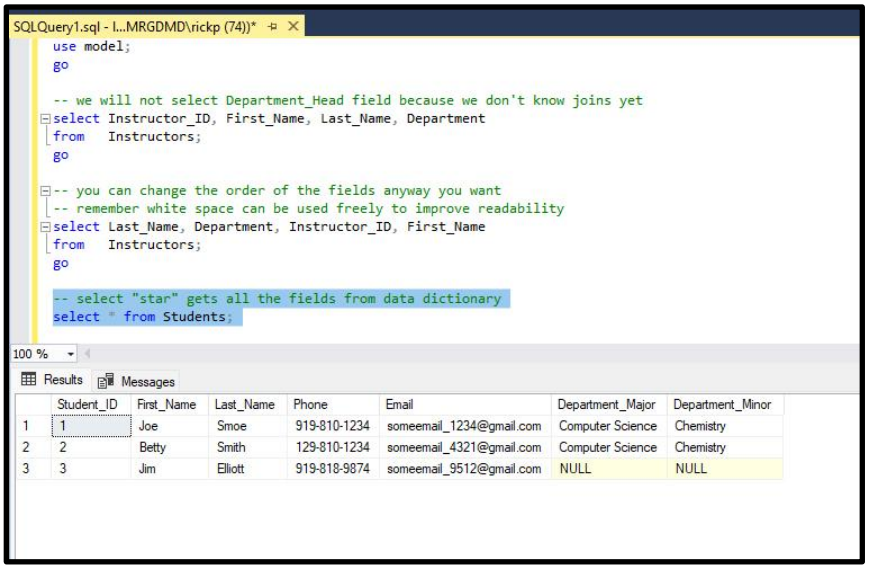


Figure 19B.3: Reorder Field List



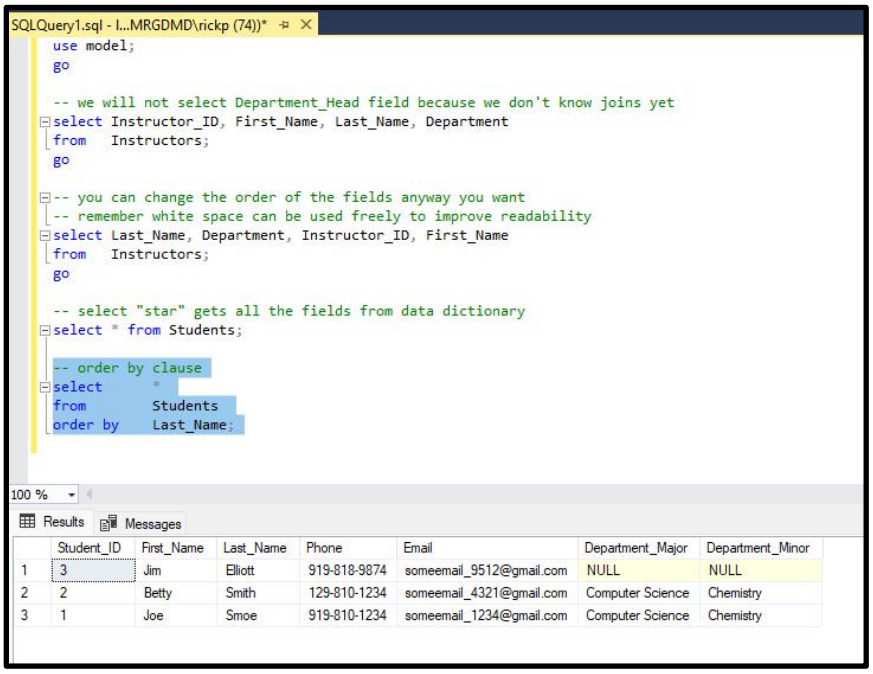
This next example you have encountered previously. The syntax “select *” which is typically read as “select star” means give me every field from the specified table(s). (Figure 19B.4) The database knows which fields are in a table via the data dictionary. The data dictionary is a schema maintained by the database which contains information about the other schemas maintained by that same database. The name of that schema in SQLServer is msdb. Never modify information within the msdb schema or you will almost certainly mess things up. Reading information from the msdb schema is fine, but never do updates and/or deletes in there.

Figure 19B.4: Select Star



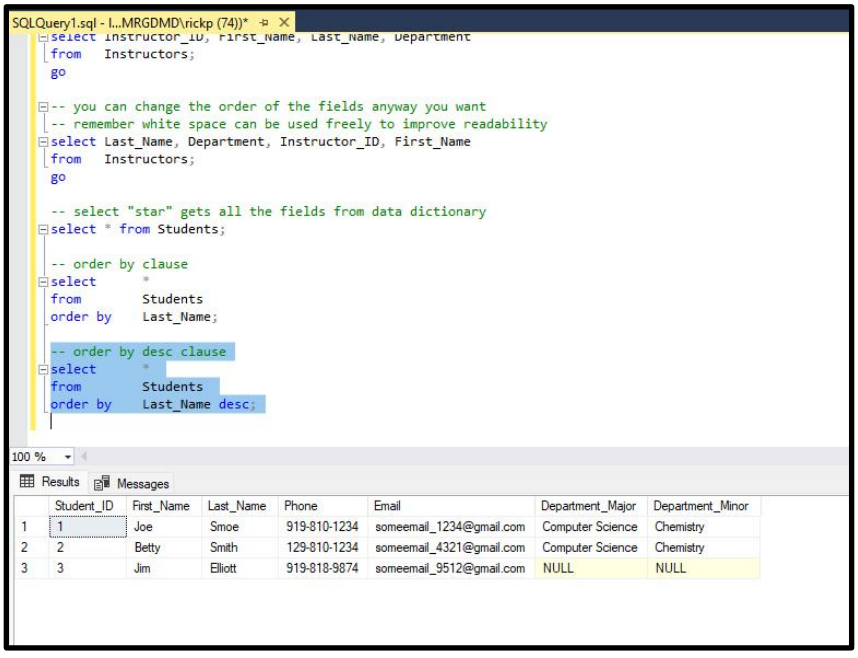
In our next example we introduce the order by clause. As the name implies, the order by clause determines the order of the rows returned to the caller. In our example, (Figure 19B.5) the rows are returned in ascending alphabetical order by Last Name. If no order by clause is included in a query the rows are typically returned in primary key order but never bet the farm on that as it is not always true. If you need the rows returned in a specific order, always use the order by clause.

Figure 19B.5: Order By Clause



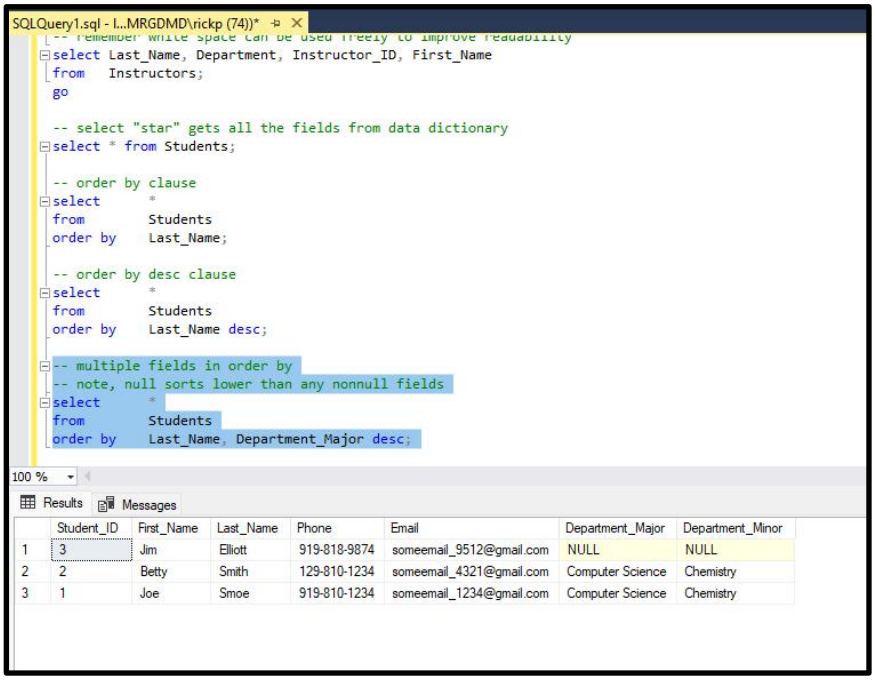
In our next example we implement a descending order by clause using the keyword desc. (Figure 19B.6) Did you notice the null value fields?

Figure 19B.6: Order By Desc



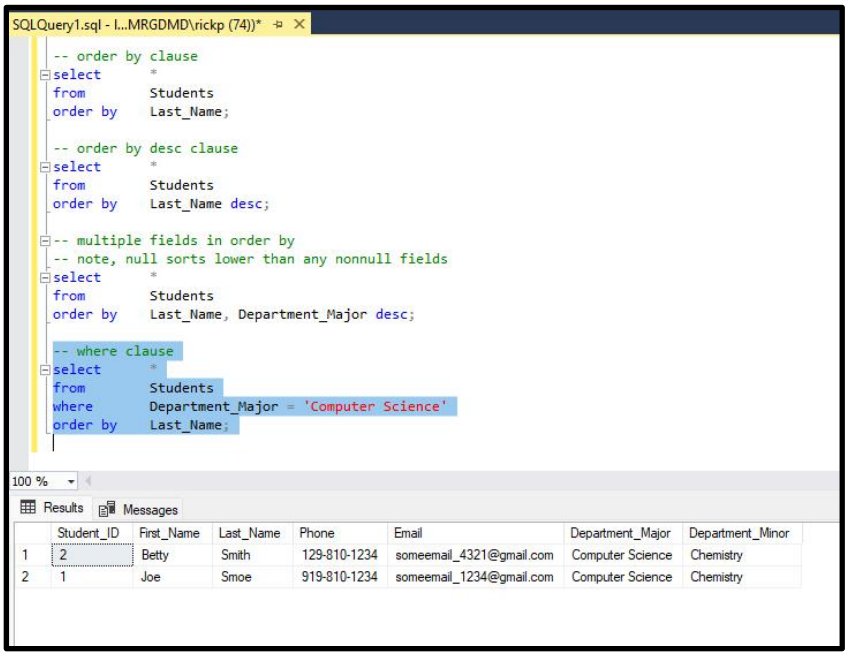
In our next example we order by two fields, one of which is ascending and the other descending. Sorts may occur to any depth and in any combination of ascending and descending values. Please also note that null value fields sort lower than any nonnull value. (Figure 19B.7)

Figure 19B.7: Multiple Order By Fields with Null Value Fields



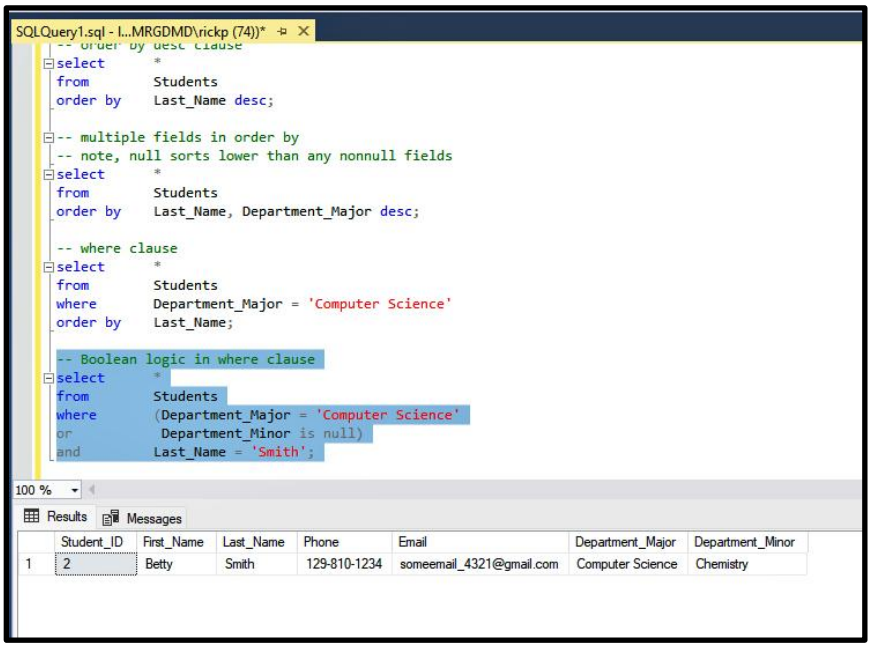
In this next example we introduce the where clause. The where clause restricts the number of rows returned to the caller based on some form of Boolean logic. (Figure 19B.8)

Figure 19B.8: Where Clause



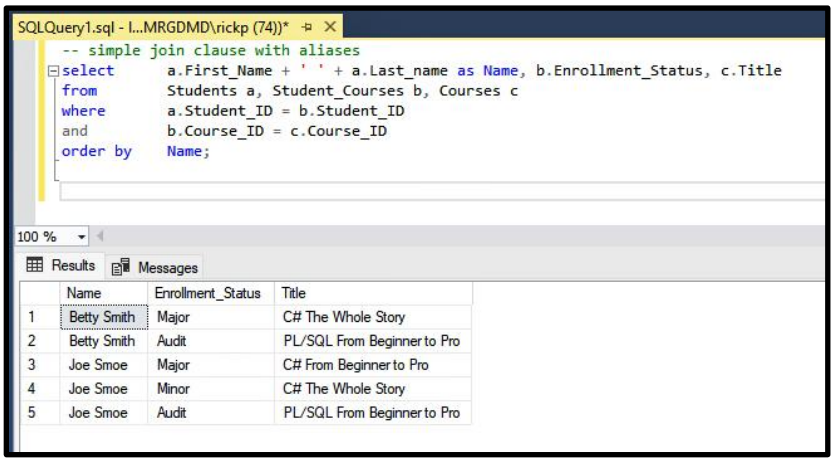
You can use any type of arithmetic, algebraic or Boolean logic in your where clause to restrict the rows returned by a query. In this example we have used both the OR and AND operators to determine which rows are returned to our output set. (Figure 19B.9)

Figure 19B.9: Advanced Where Clause



In our next example we step up our game considerably. Here we are drawing information from three different tables, the Students, Student_Courses and Courses tables. (Figure 19B.10) Each of these tables is being joined, yes that is the proper term, via their common primary key segments. In addition, we have included both table and field aliases in this example. As the name implies, an alias is used to represent a table or field in a query. The concatenation of first and last names is aliased by the "Name" value and each of the tables involved in the query have aliases as well. Table aliases simply make typing long table names easier. Field aliases are usually associated with some sort of manipulation on the fields, e.g. our concatenation of the first and last name fields.

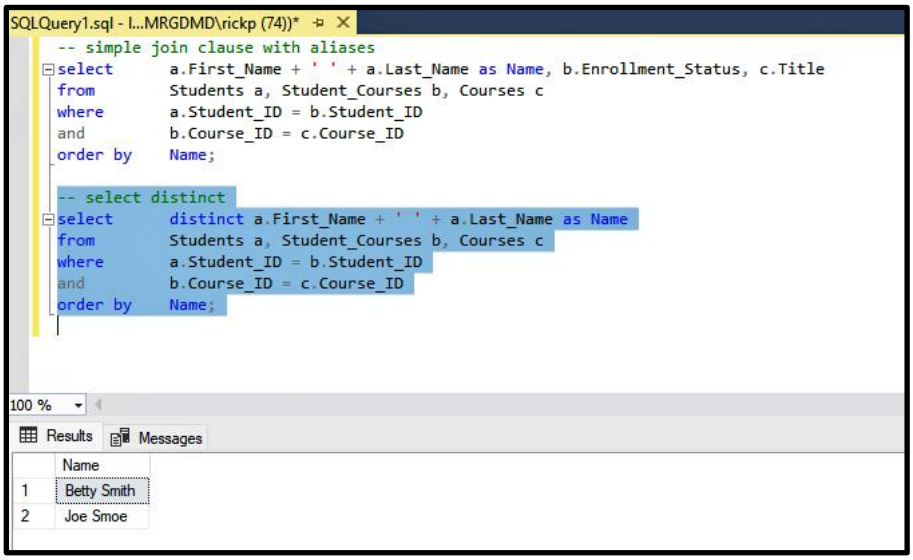
Figure 19B.10: Joins and Aliases



A valid question at this point is how many tables can be associated via joins. Simple answer, as many as it takes to get the result set of rows you need. As the complexity of a schema increases, so do the complexity of query joins. You have over 1400 tables in your schema you are going to see plenty of joins across 8-10 tables.

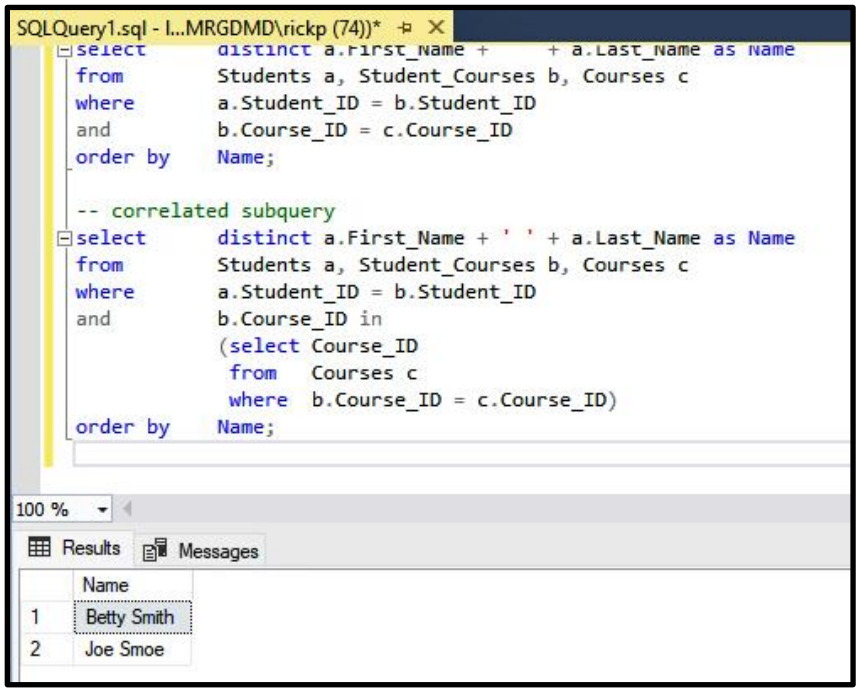
Our next example includes the keyword distinct. It runs the same query we just ran but only asks for the unique rows to be returned, i.e. the distinct rows. Pretty straight forward stuff but you will use it every day of your programming life. (Figure 19B.11)

Figure 19B.11: Distinct



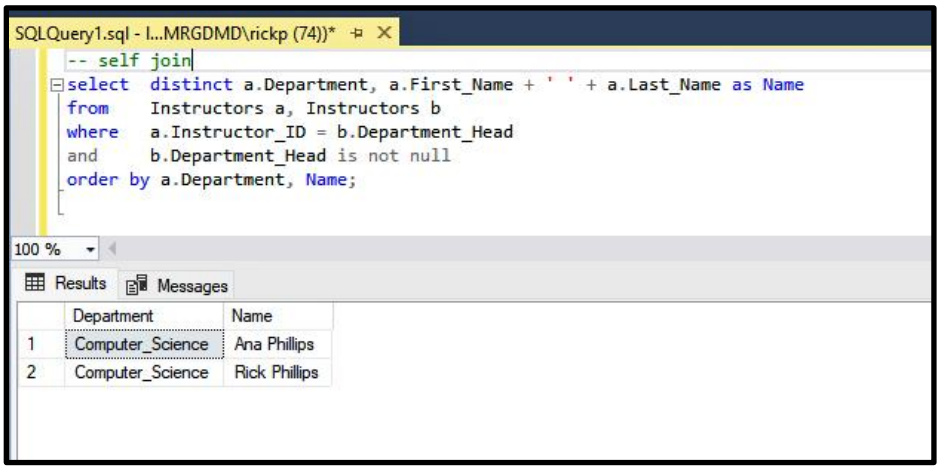
Our next example includes a subquery, i.e. a query embedded within another query. (Figure 19B.12) This subquery is known as a correlated subquery because the inner and outer queries are joined. If the subquery is not joined to the outer query the query is known as an uncorrelated subquery. Uncorrelated subqueries are very rare and typically only utilized in realizing an existence test, i.e. does this row exist within this query. Please note that nearly all correlated subqueries can be rewritten as regular joins. This subquery was just used in our previous example as a regular join.

Figure 19B.12: Subquery



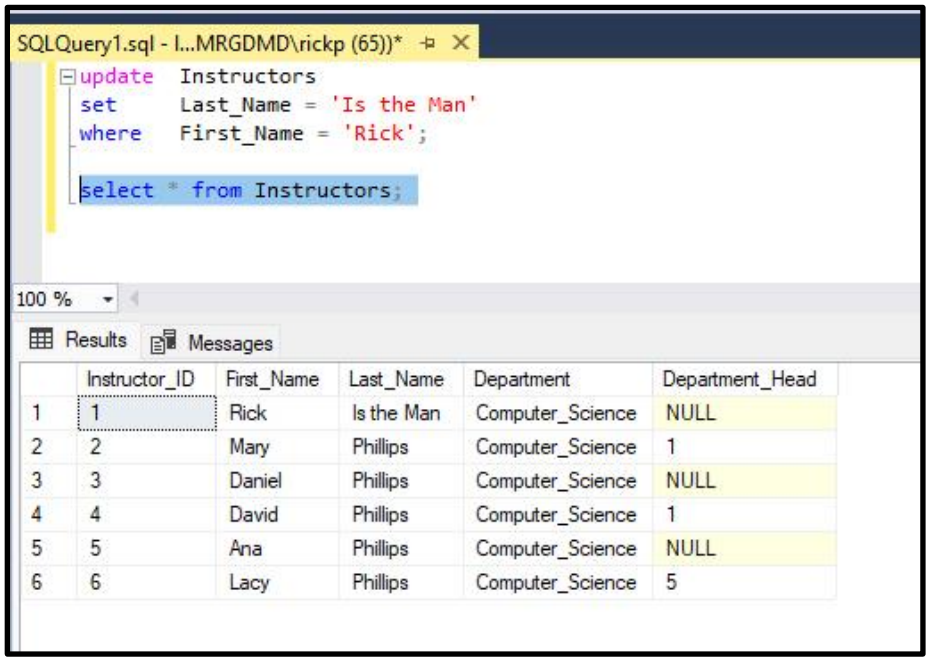
Our next example demonstrates a self-join wherein a table is joined to itself. Self-joins allow for the creation of hierarchies of data within an otherwise flat relational world. (Figure 19B.13)

Figure 19B.13: Self-Join



We next move onto the update syntax. (Figure 19B.14) Again the syntax here is super easy. If there are multiple fields to-be-updated simply separate them with a comma as shown here. (Figure 19B.15)

Figure 19B.14: Update



	Instructor_ID	First_Name	Last_Name	Department	Department_Head
1	1	Rick	Is the Man	Computer_Science	NULL
2	2	Mary	Phillips	Computer_Science	1
3	3	Daniel	Phillips	Computer_Science	NULL
4	4	David	Phillips	Computer_Science	1
5	5	Ana	Phillips	Computer_Science	NULL
6	6	Lacy	Phillips	Computer_Science	5

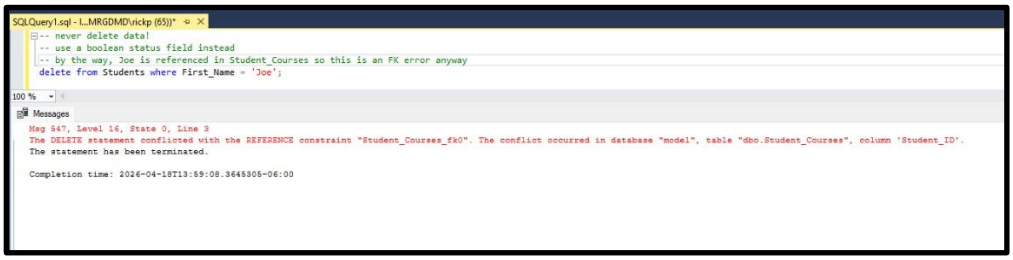
Figure 19B.15: Update Multiple Fields

```
-- update multiple fields
update  Instructors
set     Last_Name = 'Is the Man',
        Department = 'Chemistry'
where   First_Name = 'Rick';
```

In our final example we demonstrate the use of the keyword delete. (Figure 19B.16) If you are taking a certification test you need to know this syntax otherwise ignore this syntax because you should never even try to delete information within a database. Instead, set a boolean status field in each row and instead of deleting information set that status flag to false to indicate that the row is temporarily out of commission. Why did I say temporarily? Because everytime you physically delete rows in a database somebody is going to come back to you asking for them to be reinstated because some sort of mistake was made. By the way, deletes in a big database are often very difficult to realize anyway because of referential integrity constraints as shown here.

Net, use a status field instead of true deletes and you will always be able to recover from user mistakes.

Figure 19B.16: Delete (Don't Do It!)



Okay, that wraps up our discussion of SQL. The syntax is super easy but be careful with the joins to ensure you get all the key segments matched up. A note from the experienced here, if you are getting duplicate rows in your output, you almost certainly have missed some joins, so go back and rethink the query. Yes, that is the voice of scar tissue speaking. You do this long enough you are going to make mistakes. By the way, both Oracle and SQL Server support a rowcount maximum limit feature. If you are working with hundreds of thousands of possible output rows consider testing with a rowcount maximum limit first. Again, the voice of scar tissue speaking.

See you for our next lesson when we start implementing SQL within our C# code. Should be great fun.